

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
Test Summary Report
Versione 1.3



Data: 12/01/2026

Progetto: MediBook	Versione: 1.3
Documento: TSR	Data: 12/01/2026

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Lelio Crisci
--------------------	--------------

Revision History

Data	Versione	Descrizione	Autore
12/01/2026	1.0	Risultati di JUnit	Lelio Crisci
12/01/2026	1.1	Risultati Selenium	Samuele Colucci
12/01/2026	1.2	Riepilogo del Testing	Christian Casale

1. Introduzione	4
1.1. Scopo	4
1.2. Riferimenti	4
2. Risultati di JUnit-Mockito (Unit Testing)	4
2.1. Feature Testate	4
2.2. Panoramica dei risultati del test delle classi	4
2.2.1. Errori/Faliure nel Testing	4
2.2.2. Testing	5
• GestionePrenotazionTest	5
• GestioneRefertiTest	5
• GestioniUtenzaTest	6
3. Risultati di JUnit (Integration Testing)	7
3.1. Feature Testate	7
3.2. Panoramica dei risultati del test delle classi	7
3.2.1. Errori/Faliure nel Testing	7
3.2.2. Testing	7
• GestionePrenotazioneIntegrationTest	7
• GestioneRefertiIntegrationTest	8
• GestioniUtenzaIntegrationTest	8
4. Risultati di Selenium (System Testing)	9
4.1. Feature Testate	9
• Autenticazione (UC1)	9
• Registrazione Account (UC2)	10
• Prenotazione Visita (UC3)	10
• Modifica Prenotazione (UC4)	11
• Gestione Visita (UC6)	11
• Primo Accesso con Cambio Password (UC8)	12
• Compilazione Referto (UC9)	12
5. Riepilogo del Testing	14

Test Summary Report

1. Introduzione

Questo documento ha lo scopo di riportare i risultati dell'esecuzione dei test case relativi al sistema Medibook.

In particolare, sono stati testati i sottosistemi del layer di business e persistenza utilizzando JUnit e successivamente è stato effettuato il System Testing con Selenium.

1.1. Scopo

Lo scopo di questo documento è quello di fornire una presentazione dei casi di test. I vari membri del team si sono impegnati nel verificare che le singole unità abbiano il comportamento atteso.

1.2. Riferimenti

- **Documento dei Requisiti (RAD - Requirements Analysis Document):** Specifica le funzionalità (UC) e i requisiti non funzionali a cui i test fanno riferimento.
- **Documento di Architettura (ODD - Object Design Document):** Fornisce i dettagli sulla struttura del sistema, utili per comprendere i test d'unità effettuati con JUnit.
- **Test Case Specification (TCS):** Il documento che contiene la descrizione dettagliata di ogni test case eseguito (ID, input, oracolo e passaggi), di cui questo report sintetizza i risultati.

2. Risultati di JUnit-Mockito (Unit Testing)

Di seguito sono riportati i risultati dei test d'unità

2.1. Feature Testate

- GestioneMedico
- GestionePrenotazione
- GestioneReferti
- GestioniUtenza

2.2. Panoramica dei risultati del test delle classi

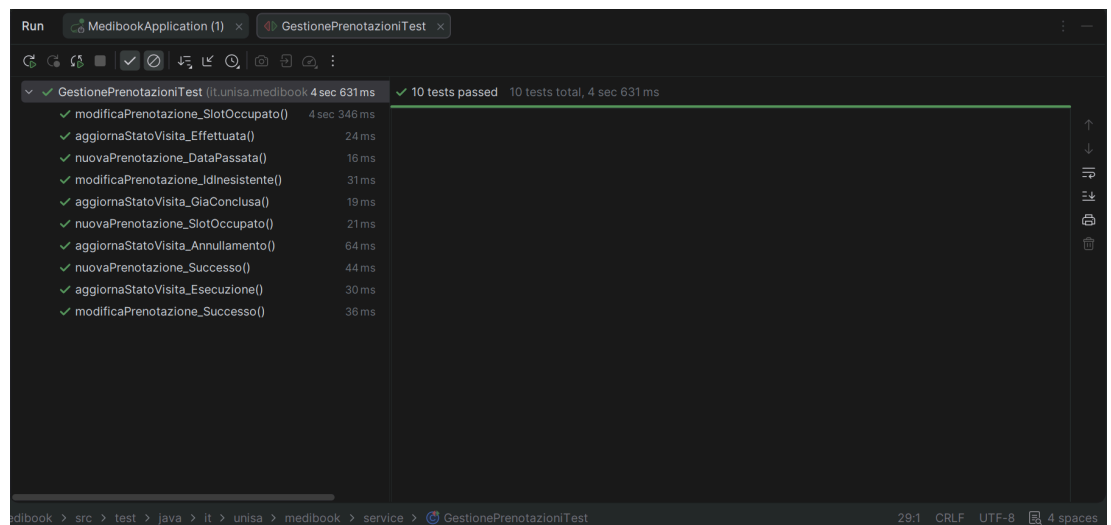
2.2.1. Errori/Faliure nel Testing

Nelle seguenti classi sono stati riscontrati errori e/o faliure:

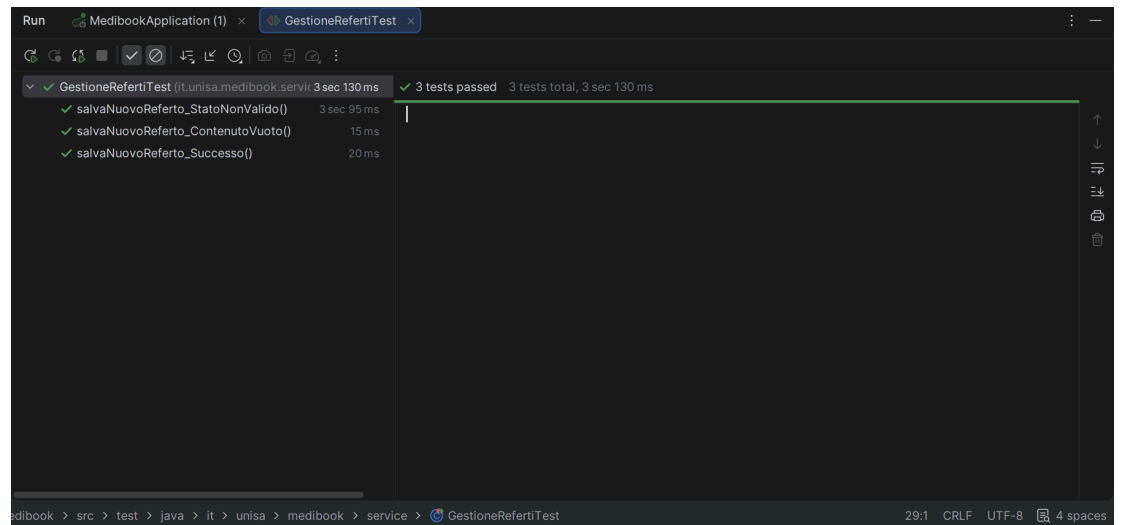
Nome Classe	Nome Metodo
GestionePrentoazione	aggiornaStatoVisita(Integer id,String nuovoStato)
GestioneReferti	salvaNuovoReferto(Integer prenotazioneId,String contenuto)

2.2.2. Testing

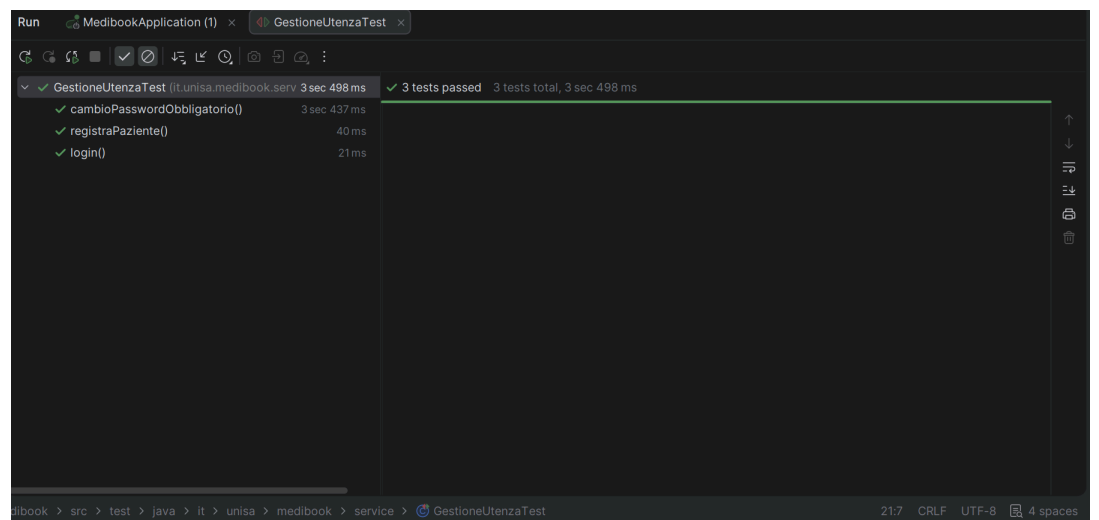
- GestionePrenotazionTest



- **GestioneRefertiTest**



- **GestioniUtenzaTest**



Il risultato dei test può essere visibile nel package:
“MediBook\Medibook\src\test\java\it\unisa\medibook\service”

3. Risultati di JUnit (Integration Testing)

Di seguito sono riportati i risultati del test di integrazione fatto sul sistema

3.1. Feature Testate

- GestioneMedico
- GestionePrenotazione
- GestioneReferti
- GestioniUtenza

3.2. Panoramica dei risultati del test delle classi

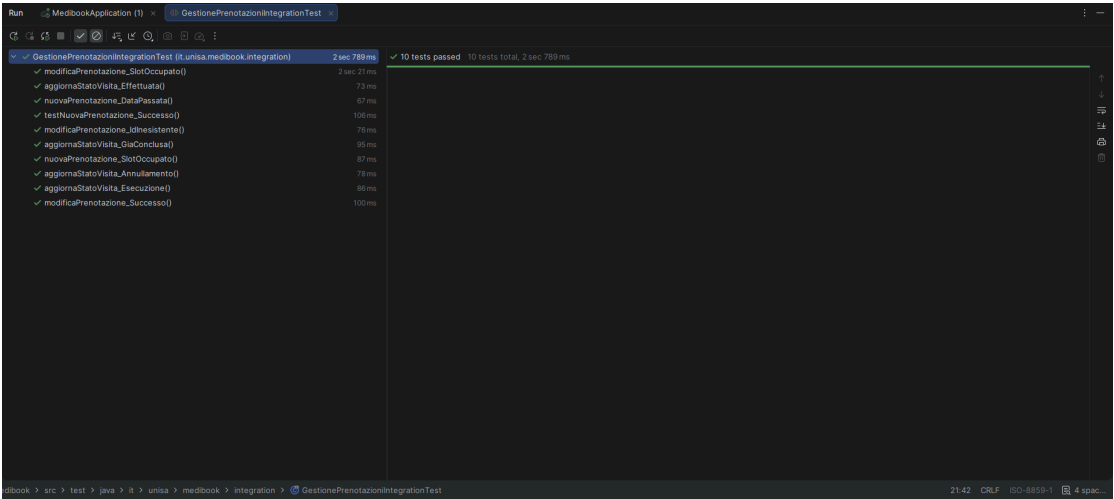
3.2.1. Errori/Faliure nel Testing

Nelle seguenti classi sono stati riscontrati errori e/o faliure:

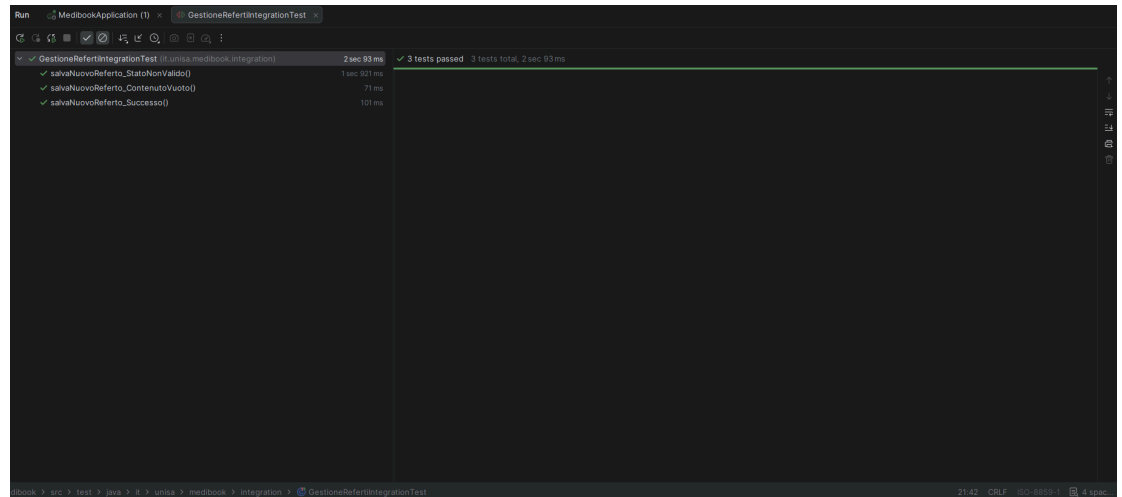
Nome Classe	Nome Metodo
GestionePrentoazione	aggiornaStatoVisita(Integer id,String nuovoStato)
GestioneReferti	salvaNuovoReferto(Integer prenotazioneId,String contenuto)

3.2.2. Testing

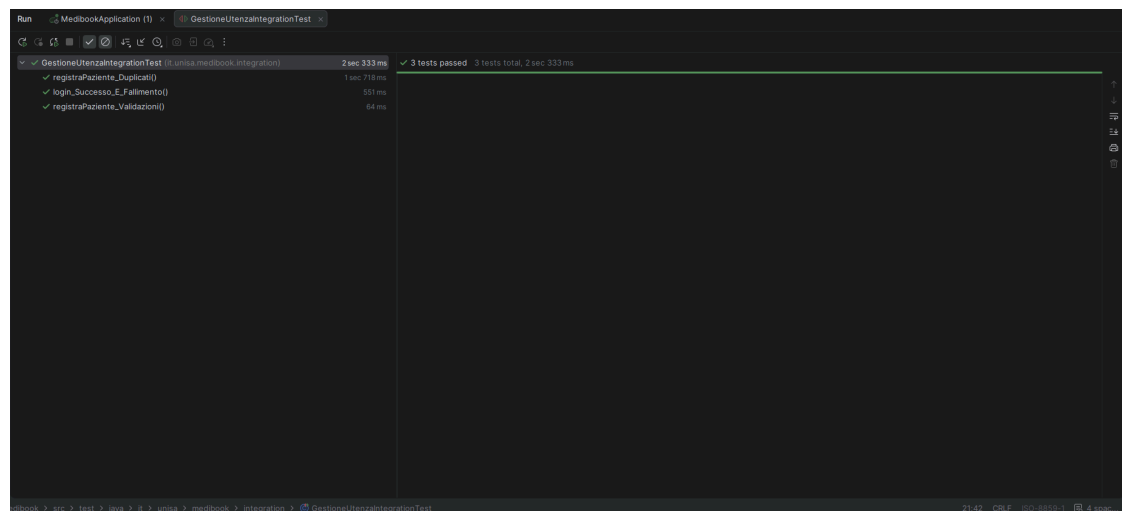
- GestionePrenotazioneIntegrationTest



- **GestioneRefertiIntegrationTest**



- **GestioneUtenzaIntegrationTest**



Il risultato dei test può essere visibile nel package:
“MediBook\Medibook\src\test\java\it\unisa\medibook\integration”

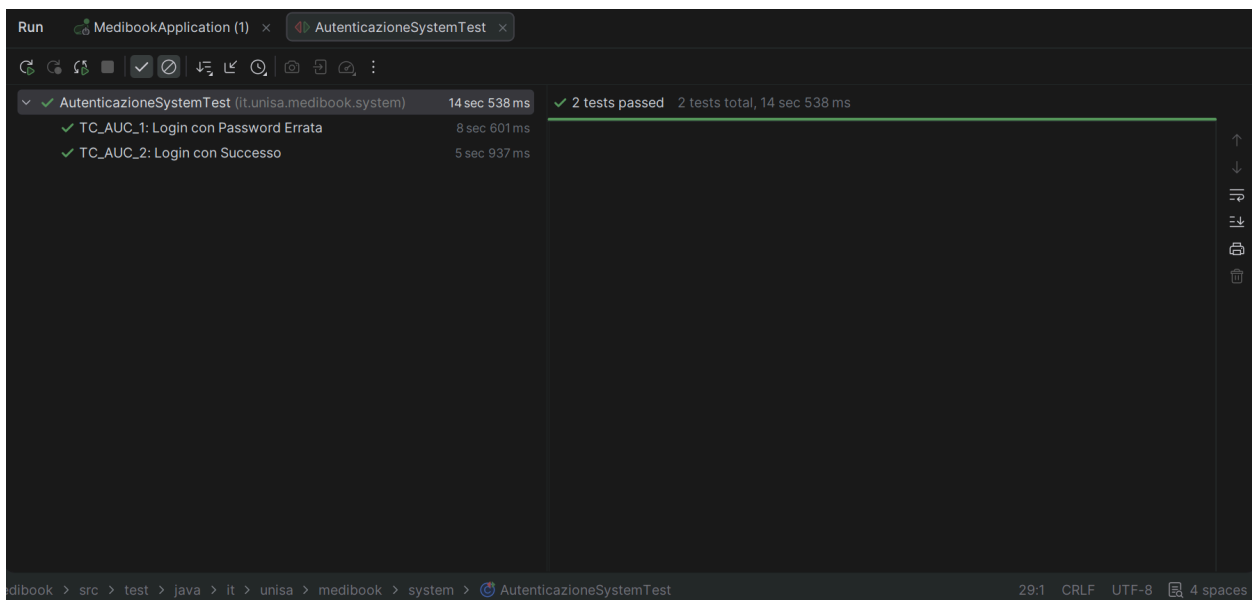
4. Risultati di Selenium (System Testing)

Di seguito è riportato il risultato del testing di sistema ottenuto mediante l'utilizzo di selenium.

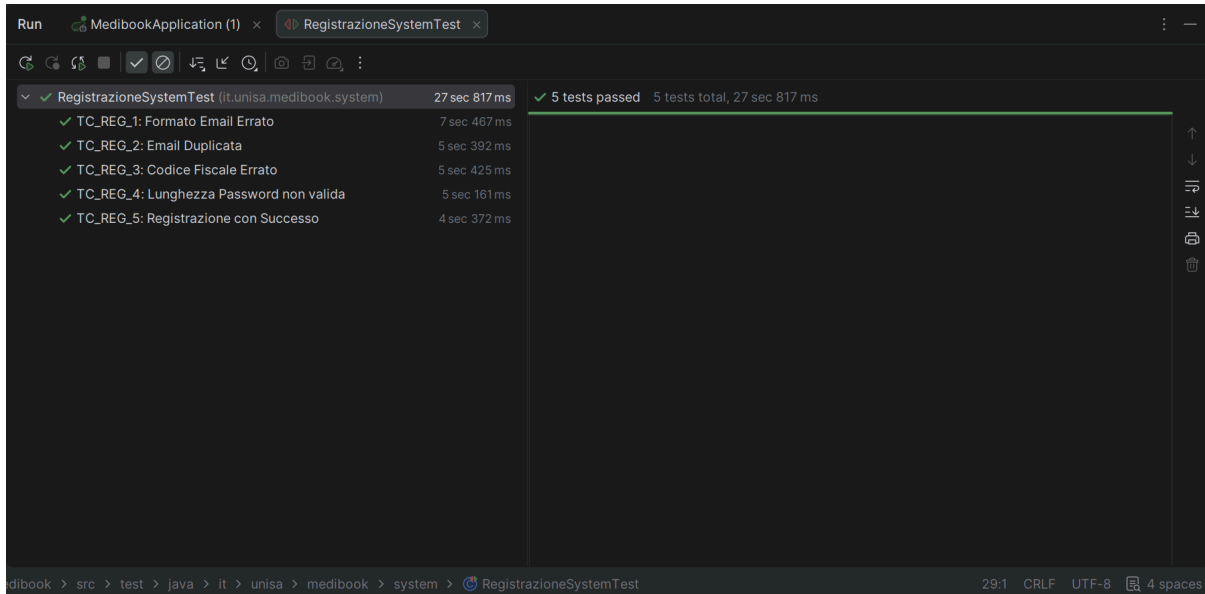
4.1. Feature Testate

Il Testing di sistema si propone di eseguire i test case pianificati nel documento TCS, in seguito l'elenco dei case test testati:

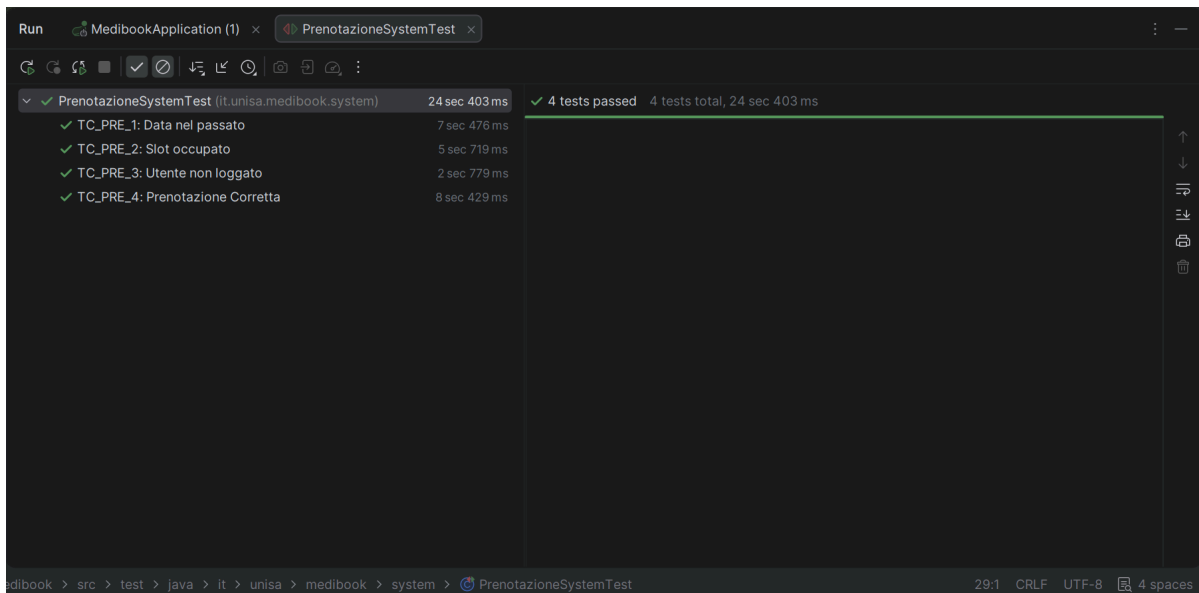
- **Autenticazione (UC1)**
 - **TC_AUC_1** Login con Password Errata
 - **TC_AUC_2** Login con Successo



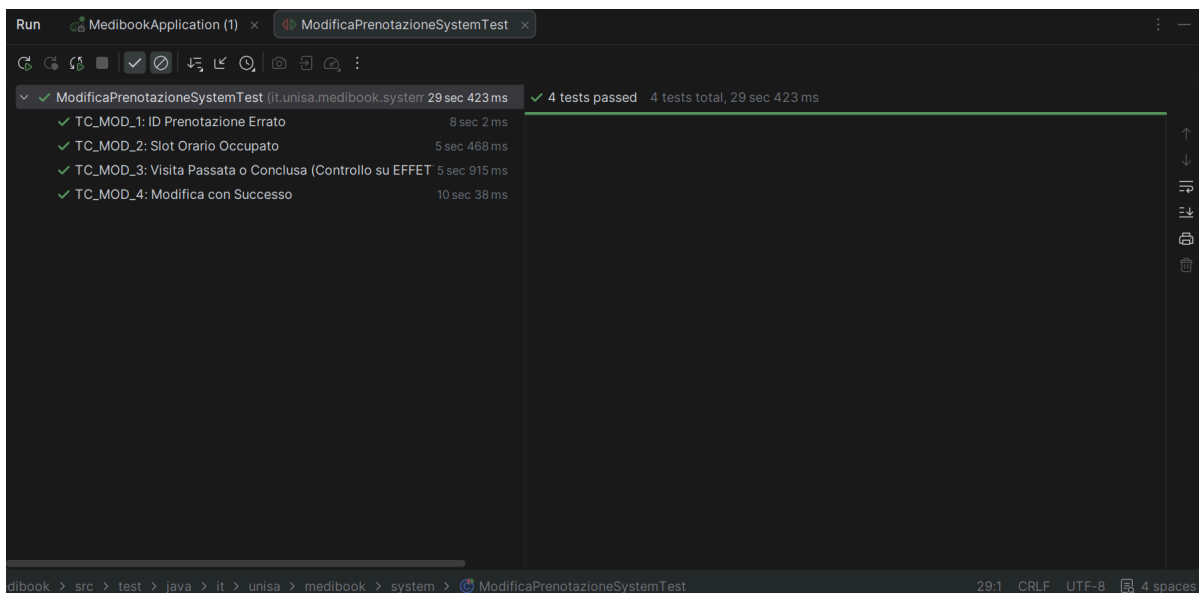
- **Registrazione Account (UC2)**
 - **TC_REG_1** Formato Email Errato
 - **TC_REG_2** Email Duplicata
 - **TC_REG_3** Codice Fiscale Errato
 - **TC_REG_4** Lunghezza Password non valida
 - **TC_REG_5** Registrazione con Successo



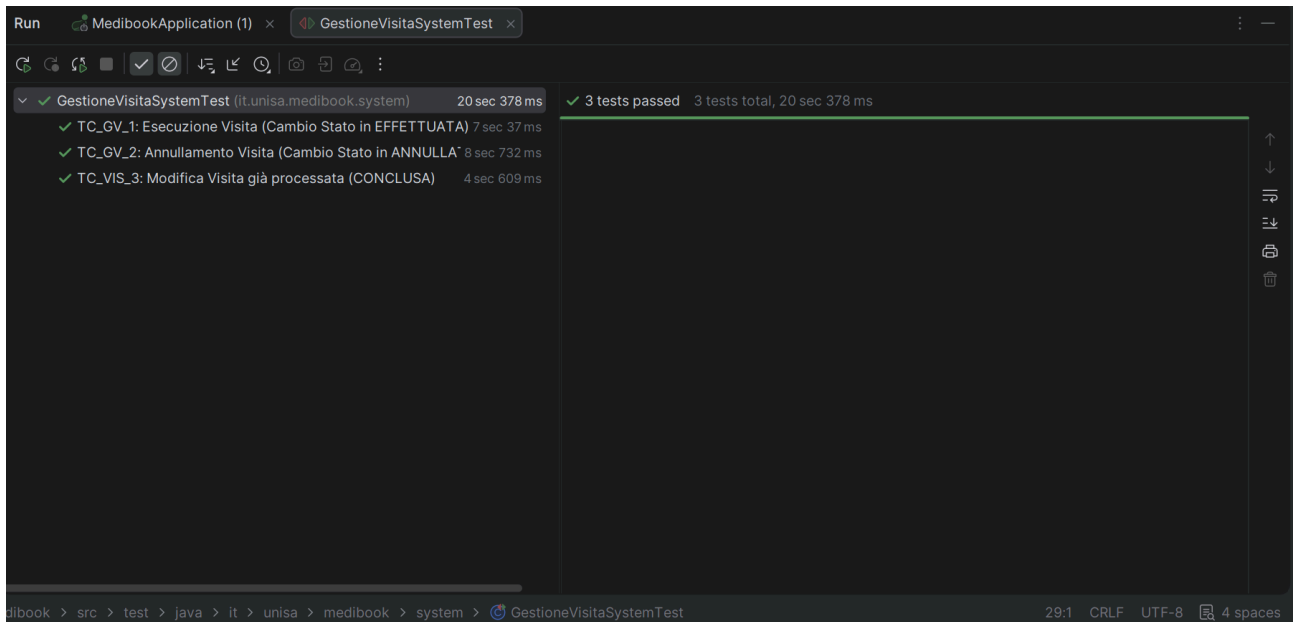
- **Prenotazione Visita (UC3)**
 - **TC_PRE_1** Data nel passato
 - **TC_PRE_2** Slot occupato
 - **TC_PRE_3** Utente non loggato
 - **TC_PRE_4** Prenotazione Corretta



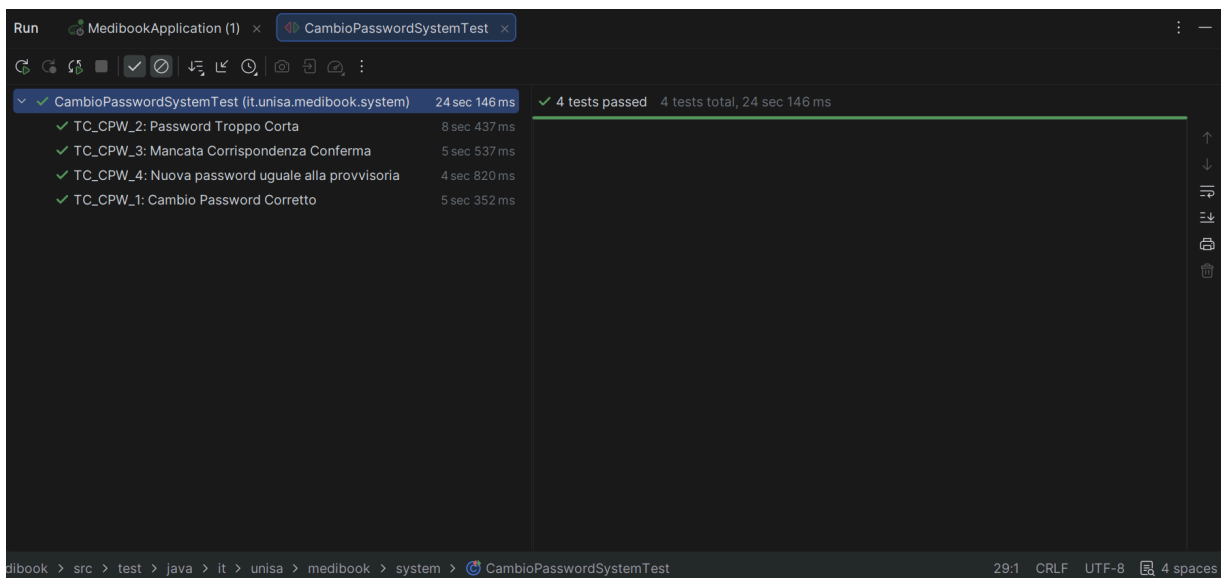
- **Modifica Prenotazione (UC4)**
 - **TC_MOD_1** ID Prenotazione Errato
 - **TC_MOD_2** Slot Orario Occupato
 - **TC_MOD_3** Visita Passata
 - **TC_MOD_4** Modifica con Successo



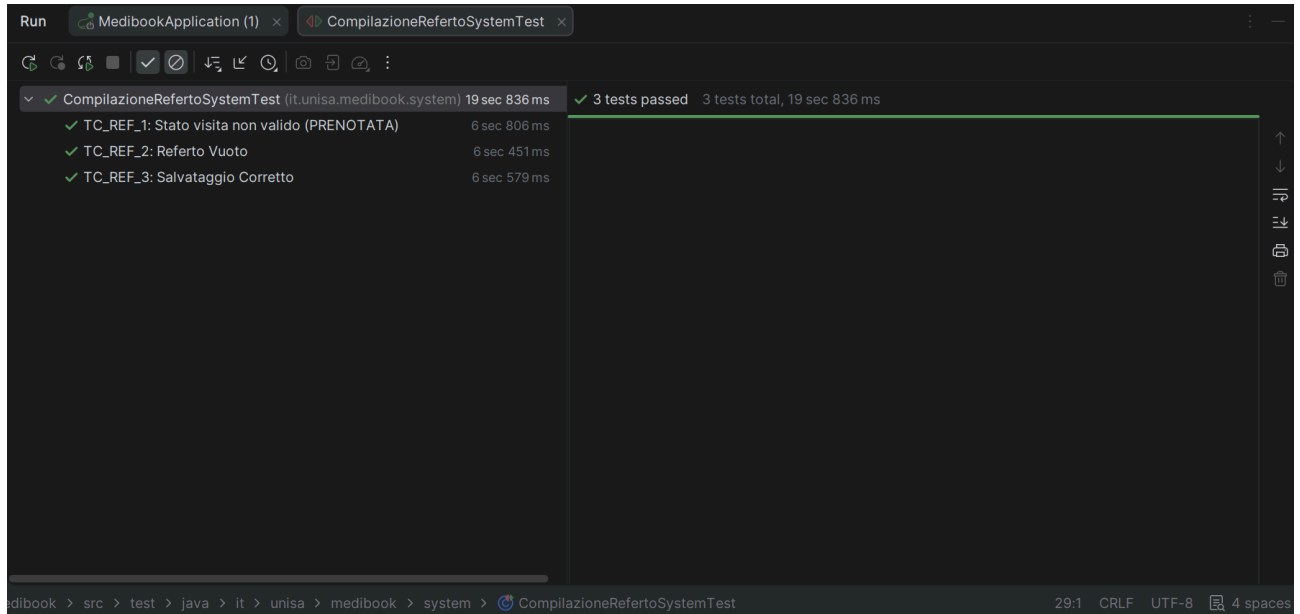
- **Gestione Visita (UC6)**
 - **TC_VIS_1** Esecuzione Visita
 - **TC_VIS_2** Annullamento Visita
 - **TC_VIS_3** Modifica Visita già processata



- **Primo Accesso con Cambio Password (UC8)**
 - **TC_CPW_1** Cambio Password Corretto
 - **TC_CPW_2** Password Troppo Corta
 - **TC_CPW_3** Mancata Corrispondenza Conferma
 - **TC_CPW_4** Nuova password uguale alla provvisoria



- **Compilazione Referto (UC9)**
 - **TC_REF_1** Stato visita non valido
 - **TC_REF_2** Referto Vuoto
 - **TC_REF_3** Salvataggio Corretto



Il risultato dei test può essere visibile nel package:

“MediBook\Medibook\src\test\java\it\unisa\medibook\system”

5. Riepilogo del Testing

In conclusione, l'attività di verifica effettuata sul sistema MediBook ha permesso di convalidare le funzionalità critiche individuate nei casi d'uso principali. L'approccio di testing multi-livello ha prodotto i seguenti esiti:

- **Testing d'Unità (JUnit, Mockito):** L'esecuzione dei test sulle classi di business e persistenza ha evidenziato un'elevata affidabilità generale del codice. Tuttavia, sono state identificate delle criticità puntuali nelle classi Gestione Prenotazione (metodo `aggiornaStatoVisita`) e Gestione Referti (metodo `salvaNuovoReferto`). Tali anomalie richiedono un intervento di bug-fixing per garantire la piena conformità ai requisiti logici del sistema.
- **Testing di Integrazione (JUnit):** L'attività di verifica si è concentrata sulla corretta cooperazione tra i diversi componenti del sistema, validando in particolare l'interazione tra la logica di business e il livello di persistenza dei dati. I test hanno confermato la stabilità delle transazioni verso il database e il rispetto dei vincoli di integrità referenziale, assicurando che i dati vengano salvati, recuperati e aggiornati correttamente attraverso i vari strati dell'architettura senza l'ausilio di framework esterni (Mockito). Tuttavia, sono state identificate delle criticità puntuali nelle classi Gestione Prenotazione (metodo `aggiornaStatoVisita`) e Gestione Referti (metodo `salvaNuovoReferto`).
- **Testing di Sistema (Selenium):** I test di sistema condotti sulle interfacce utente hanno riportato un esito positivo. Nello specifico:
 - Le procedure di Autenticazione e Registrazione gestiscono correttamente sia i casi di successo che i vari scenari di errore (formati errati, duplicati).
 - I moduli di Prenotazione, Modifica, e Gestione Visita hanno dimostrato di rispettare i vincoli temporali e di stato previsti.
 - Le funzionalità di Cambio Password e Compilazione Referto sono risultate robuste e conformi alle specifiche